

Huffman File Compressor

Report

Programming Assignment 1 Track B

Ethan Goldman

What I built

This project aims to implement a working lossless file compressor built around static Huffman coding.

What is Huffman coding

Huffman coding compresses files of any types by building a prefix-free binary code determined by processing byte frequencies. This type of compression is able to compress any file type as it compresses all the bytes in the file, no matter what the file type is.

At a high level, Huffman coding works like this:

1. Count frequencies. For BANANA, A appears 3 times, N 2 times, and B 1 time.
2. Build a tree. Make one leaf per symbol. Repeatedly join the two least frequent nodes under a new parent whose frequency is their sum, until one tree remains.
3. Assign codes. Label left branches 0 and right branches 1. Each symbol's code is the path from the root to its leaf. In the BANANA example: A=0, B=10, N=11.
4. Encode. Replace each symbol with its code: BANANA becomes 100110110. Pack those bits into bytes, and save the tree so the codes can be recovered later.
5. Decode. Read bits while walking the saved tree. On reaching a leaf, output its symbol and return to the root.

The central idea is that frequent symbols usually end up closer to the root, giving them shorter codes. This in turn means that more frequent symbols (or bytes) take up less physical space than less frequent ones (with longer codes), which in turn leads to a smaller file size.

The compressed file stores a header, which includes the original file size, number of symbols (bytes), and number of bits (as last bytes might not always be complete and have trailing 0s). It then stores the tree, which is used in decoding to allow the decoder to understand what bit representation corresponds to which symbol. Simpler implementations can use a frequency table, storing each byte and their counts, but this leads to a larger header size, and slower decoding. Instead of just walking the tree (which is quicker the more frequent the symbol appears), with stored frequencies, it would need to rebuild the tree, then walk it to decode.

The need for a header does mean that smaller or already compressed files can lead to little improvement, no improvement, or even an increase in file size. Additionally, files and file formats (MP4, PDF, etc.) that are already heavily compressed see little improvement when compressing through Huffman coding (more on this later).

My implementation

I built the algorithm in C++17 to compress or decompress one file at a time. Running the program opens a text-based terminal menu where the user chooses what to do and enters the file paths. In the program:

- The input is treated as bytes, so all file types are supported and follow the same path.
- The compressor counts all 256 possible byte values, puts one leaf per occurring value in a minimum-priority queue, and repeatedly merges the two lightest trees.
- Every parent stores the sum of its children's frequencies.
- A left edge is 0 and a right edge is 1; the path from the final root to a leaf becomes that byte's code. A lone distinct byte receives code 0.

Code organisation

- `src/huffman.cpp` defines frequency counting, tree construction, code generation and small bit-string helpers used in tests. Node children are indices into a vector, not owning pointers.
- `src/tree_io.cpp` saves and loads the tree. `src/archive.cpp` writes the HUF1 archive and streams compression and decompression. Compression reads the input twice but does not hold the whole file or encoded payload in memory.
- `src/file_ops.cpp` handles filesystem checks and temporary outputs. `src/main.cpp` provides the interactive menu. `tests` contains GoogleTest checks; `bench` contains the gzip comparison harness.

Archive representation

Each HUF1 file starts with a 22-byte header: the four ASCII bytes HUF1, an eight-byte original size, a two-byte distinct-symbol count and an eight-byte meaningful payload-bit count. Integers are stored little-endian (least significant first). A preorder tree follows: an internal node is marker 00, while a leaf is marker 01 followed by its byte value. The payload packs bits most-significant-first, with zero padding on the right of the final byte. The saved tree and bit count let a different run decode the file without the original input or its frequency table. An empty input has no tree or payload and produces a 22-byte archive.

This is a deliberately simple format. There is one static tree per file, no dictionary stage, no raw-data fallback and no checksum. The decoder rejects malformed headers and trees, truncation, inconsistent lengths, nonzero padding and trailing bytes. It cannot detect every payload bit flip if the altered bits still form a valid stream with the expected output length.

For n input bytes, k active byte values and B meaningful encoded bits, compression takes time proportional to the input scan and bit output, plus $k \log k$ priority-queue work and generation of the code paths. Decompression scans the saved tree and B bits while emitting the original bytes. Here k is at most 256, so the tree and code table have bounded size independent of file length. File content remains on disk and is processed as streams.

The tool

Interface and safeguards

Running the compiled program without arguments opens a terminal menu with Compress, Decompress, Inspect, Help and Exit. Compression proposes an input path with `.huf` appended; decompression removes a final `.huf` or proposes `.out`. A blank input cancels, while a blank output accepts the suggestion. Literal paths with spaces work. The program asks before replacing an existing output and rejects input/output aliases and output symbolic links. It writes into a temporary directory beside the destination and renames the completed file only after a successful operation and a failed decode does not replace an existing file. The alternate terminal screen restores the previous terminal contents on exit, and a live timer reports elapsed time during compression or decompression.

Inspect uses the normal decoder while discarding restored bytes. It reports archive size, original size, symbol count, header and tree size, payload bits and compression statistics without writing an extracted file. This checks archive structure and payload length, subject to the checksum limitation above.

Worked example

For the six-byte input BANANA, the constructed codes are $A = 0$, $B = 10$ and $N = 11$. The encoded sequence is 100110110: nine meaningful bits packed as 9B 00 in hexadecimal. The tree occupies eight bytes, so the 22-byte header, tree and two payload bytes make a 32-byte archive. This small example expands because metadata costs more than the bits saved. The decoder reads only nine payload bits, ignoring the seven zeros added to complete the second byte, and restores BANANA.

For a realistic menu use, selecting Compress on `test_data/pride_and_prejudice.txt` and accepting the default output produces a 446,927-byte HUF1 file from 772,386 input bytes. Inspect can validate that archive without extracting it. Decompressing to a new filename restores the original bytes; the benchmark harness confirmed an exact byte-for-byte match.

Correctness checks

The 51 passing GoogleTest cases cover byte values 0 through 255, empty and single-symbol files, exact BANANA header/tree/payload bytes, bit-padding boundaries, seeded random inputs, truncated or malformed archives, and file-safety failures. The independent expected bytes matter because a compressor and its matching decoder could share a mistake yet still round trip. The six dataset files were also restored exactly by both HUF1 and gzip in each of three benchmark repetitions. While these checks give useful evidence, they don't provide exhaustive proof over every possible file or corrupt archive.

Gzip comparison

The comparison uses six unchanged inputs: a novel as text and EPUB, GoogleTest C++ source, RFC 1951 as text and PDF, and the Python logo PNG. Their sources and SHA-256 hashes are in test_data/README.md. bench/compare.py compiles the Huffman driver with C++17 and -O2, runs it and Apple gzip with -n -6 three times per file on a M1 Pro MacBook Pro, alternates tool order, compares each restored output against its input, and saves all 36 runs in bench/results.csv. Size means the complete archive, including headers. The table below reports the fixed sizes and savings.

The timer measures each completed file-to-file subprocess, including startup and file I/O, but excludes compilation and the byte comparison. This comparison is mostly aimed towards comparing the size saving performance of these algorithms, not speed. As such, speed of these algorithms can only be compared to each other and shouldn't be used as exhaustive proof of broader performance ranking.

Table 1 Complete archive sizes and space saved

Input	Original bytes	HUF1 bytes	gzip bytes	HUF1 saved	gzip saved
Pride text	772,386	446,927	275,902	42.14%	64.28%
GoogleTest C++	266,604	166,777	62,873	37.44%	76.42%
RFC text	36,944	20,504	11,150	44.50%	69.82%
RFC PDF	56,620	56,278	47,273	0.60%	16.51%
Python logo	45,187	9,986	4,280	77.90%	90.53%
Pride EPUB	24,835,578	24,782,084	24,397,894	0.22%	1.76%

Original bytes, Huffman archive bytes and gzip archive bytes

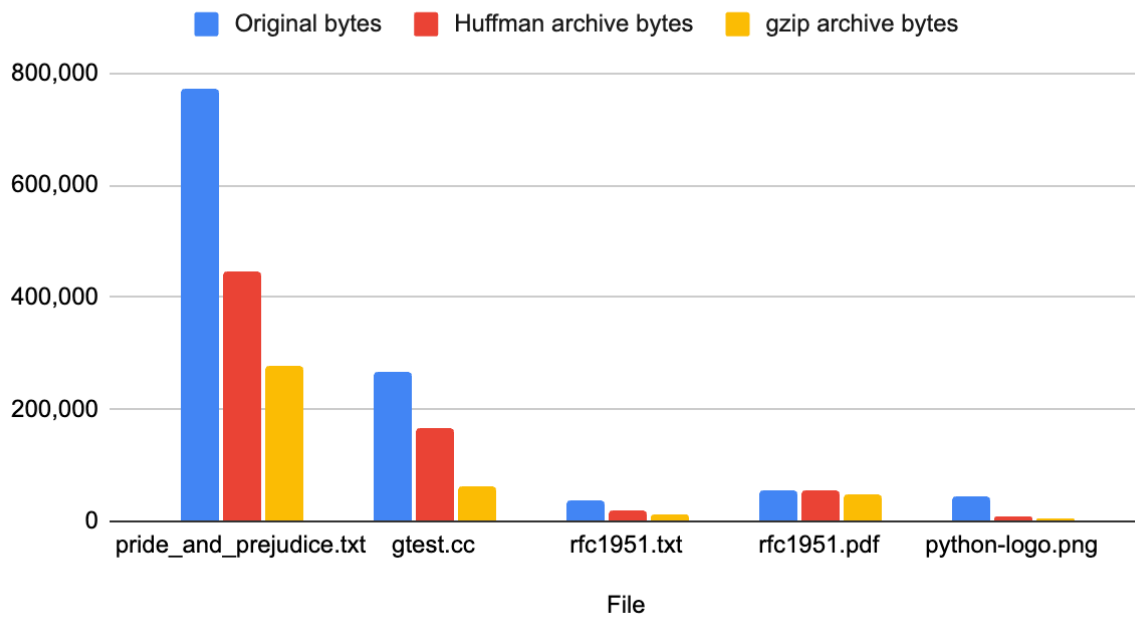


Figure 1 Original and archive sizes for the five smaller test files. The EPUB is omitted from the chart because its much larger size would make the other bars difficult to compare, its results remain in Tables 1 and 2.

Table 2 Median completed operation time in milliseconds

Input	HUF1 compress	gzip compress	HUF1 restore	gzip restore
Pride text	55.42	34.52	39.43	5.23
GoogleTest C++	22.55	9.55	16.83	3.72
RFC text	6.89	4.59	6.17	3.63
RFC PDF	9.63	4.15	8.69	3.36
Python logo	5.61	3.85	4.99	3.56
Pride EPUB	2108.34	558.07	1993.50	66.02

Gzip produced smaller files and completed compression faster on all six inputs I tested. Across these files, Huffman saved an average of 33.8% of the original size, while gzip saved 53.2%. The average compression times were about 368 ms for Huffman and 102 ms for gzip, although both averages are heavily affected by the much larger EPUB file. For that file, Huffman took about 3.78 times as long, or 278% slower. The tests show

how these two implementations performed on my dataset, not how every Huffman or gzip implementation would perform.

What I learned

What surprised me

The results actually surprised me. When first researching this algorithm, I did not expect such a significant gap between my compressor and gzip. The difference also depended heavily on the file. For the EPUB, gzip saved only about 1.55 percentage points more space than Huffman, while for the GoogleTest C++ file it saved about 38.97 percentage points more. Seeing that range made me realise that the type of redundancy in a file matters, not just its size or extension.

Additionally, the speed of the algorithms also surprised me. I understand that my implementation is far from complete and optimised and improvements could be made. I was aiming for an algorithm that was easy for me to understand and write, rather than a fully optimised one. However, seeing speed differences range from 46% slower to 278% slower still surprised me.

What surprised me the most overall was how radically different the 2 algorithms can be in terms of speed and compression ratio on both the same file, and overall on all files. I would definitely be interested in learning how gzip works and why it can compress more than Huffman in every single instance.

What did I initially get wrong

When designing the HUF1 file format, I initially thought I should store a frequency table because that was how I had learned to build the tree. I later realised the decoder does not actually need the original frequencies; it needs the tree that resulted from them. A straightforward table with an eight-byte count for each of the 256 possible bytes would take 2,048 bytes. My saved tree takes $3k - 1$ bytes for k distinct bytes, or 767 bytes even when all 256 occur. Storing the tree also means the decoder can use it immediately instead of rebuilding it. This taught me that implementing the algorithm and designing a compact file format are related, but separate problems.

What is something I understand now that I didn't before starting

This isn't completely related to my actual implementation, and I did believe this was the case beforehand, but I now understand why you cannot just infinitely, losslessly compress files and get an improvement every time. Re-compressing an already compressed HUF1 file resulted in a larger file size. This was due to the compressed file already being optimised for this algorithm and the need for another header after the compressed original header.

I also conducted an experiment compressing a Huffman file with gzip, which did produce more space savings. Interestingly, gzip alone still produced the smallest file size. I imagine that this is due to the original file having more repetitive sequences of bytes (which gzip can exploit) compared to the compressed file.

AI use

Which tools I used and what I used them for

I used a variety of OpenAI models through Codex. I primarily used 5.6 Sol (later 6 Sol when it released) for when I need a little bit of help while implementing the main algorithm, and small fixes. 6 Astra was used little bit to explain the concept of Huffman coding and break it down to me. It was also used for code review and reviewing my work and logic.

For the actual Huffman algorithm, I tried not to use or rely on AI as much as possible, so most of that was written by me. Sol was used to help with things like the UI (making the program clear the terminal on each selection), writing tests (including porting to Gtest once I realised that it was becoming too complicated to write tests manually), the HUF1 file format creation, and the test harness for benchmarking against gzip. It is hard to me to pinpoint exactly how much it assisted in each of these tasks and would say it's a light to moderate level.

At least two specific examples of where the AI was wrong, unhelpful, or misleading

Bit Packing

While implementing the `pack_bits` function, which packs 8 bits into a byte, Codex suggested to loop through every single bit, storing the bits temporarily until it hits 8 bits long, then appending the byte. I initially was very confused as to why it recommended this and was starting to second guess why I couldn't just count every 8 bits and immediately turn it into a byte. This is more efficient and simple than looping through every bit. Upon questioning Codex with this, it then confirmed that my idea is in fact simpler and to use it instead. While both implementations work and would probably not meaningfully affect performance, I preferred my own, more logical solution.

Incorrect understanding of `std::bitset<8>`

When asking how to turn a string of 8 bits into an actual byte to store in a file, Codex proposed `std::bitset<8>`. Codex claimed that `bitset` would add trailing 0s if it wasn't exactly 8 bits. My tests kept on failing when I realised that the tests where I was processing less than 8 bits, it was actually appending leading 0s (i.e. before the bits). This would in turn change the symbol entirely and would malform the encoding, as my

format requires reading the bits from left to right. This led to the discovery that bitset appends to the start, not the end. I corrected this by appending 0s to the end of the substring (if needed) and then calling bitset.

It is interesting that both of these instances happened in the same function and within a couple prompts of each other.

What I understood vs what I took on trust

I fully understand the algorithm and trust it as I had very little help from AI and coded a majority of it myself. For the parts where it was a mixture of AI and I, I carefully reviewed the code and ensured it wasn't hallucinating, cheating, or incorrect in general. For example, when I was adding tests, I would read through each test and ensure it was actually testing something useful, and it was testing it correctly. AI loves to create 100s of tests that all do and prove absolutely nothing and then reassuring itself that the code passes "all the tests". For less important parts like the UI and general usage of the program, I mostly trusted the small edits it made as they were small, specific edits, which I could test the functionality of simply (i.e. it either works or doesn't). For example, I was having trouble figuring out how to make the program switch to an alternate screen buffer, clearing the screen and redrawing the menu between prompts. I asked Codex to implement it, it did, I tested it, it worked. I therefore trusted its code as it did in fact work.